

第三章

1. 验证旋转矩阵是正交矩阵。

$\mathbf{a} = \mathbf{R}\mathbf{a}'$, $\|\mathbf{a}\|^2 = \|\mathbf{a}'\|^2$, 展开范数的平方:

$\mathbf{a}^T \mathbf{a} = \mathbf{a}'^T \mathbf{a}'$, 代入 $\mathbf{a} = \mathbf{R}\mathbf{a}'$ 得:

$(\mathbf{R}\mathbf{a}')^T (\mathbf{R}\mathbf{a}') = \mathbf{a}'^T \mathbf{R}^T \mathbf{R} \mathbf{a}' = \mathbf{a}'^T \mathbf{a}'$, 对任意向量 $\mathbf{a}'$ 都成立, 所以:
 $\mathbf{R}^T \mathbf{R} = \mathbf{I}$.

2. * 寻找罗德里格斯公式的推导过程并加以理解。

向量 $\mathbf{v}$ 沿单位旋转轴向量 $\mathbf{k}$ 旋转角度 θ 后得到的旋转向量 $\mathbf{v}_{\text{rot}}$ 的罗德里格斯公式:

$\mathbf{v}_{\text{rot}} = \mathbf{v} \cos \theta + (\mathbf{k} \times \mathbf{v}) \sin \theta + \mathbf{k}(\mathbf{k} \cdot \mathbf{v})(1 - \cos \theta)$, 推导:

$\mathbf{v}_{\text{rot}} = \mathbf{v}_{\parallel \text{rot}} + \mathbf{v}_{\perp \text{rot}}$, $\mathbf{v} = \mathbf{v}_{\parallel} + \mathbf{v}_{\perp}$,

$\mathbf{v}_{\parallel \text{rot}} = \mathbf{v}_{\parallel} = (\mathbf{v} \cdot \mathbf{k}) \mathbf{k}$,

$\mathbf{v}_{\perp \text{rot}} = \cos(\theta) \mathbf{v}_{\perp} + \sin(\theta) \mathbf{k} \times \mathbf{v}_{\perp} = \cos(\theta) \mathbf{v}_{\perp} + \sin(\theta) \mathbf{k} \times \mathbf{v}$,

由..., $\mathbf{v}_{\perp} = \mathbf{v} - \mathbf{v}_{\parallel} = \mathbf{v} - (\mathbf{k} \cdot \mathbf{v}) \mathbf{k}$,

...

3. 验证四元数旋转某个点后, 结果是一个虚四元数 (实部为零), 所以仍然对应到一个三维空间点, 见式 (3.33)。

4. 画表总结旋转矩阵、轴角、欧拉角、四元数的转换关系。

以下是 **欧拉角**、**轴角**、**旋转矩阵** 和 **四元数** 的转换关系表:

起点/ 终点	欧拉角	轴角	旋转矩阵	四元数
欧拉角	-	1. 转为旋转矩阵 2. 从旋转矩阵转轴角	组合单轴旋转矩阵 $R = R_z R_y R_x$	1. 转为旋转矩阵 2. 从旋转矩阵转四元数
轴角	1. 转为旋转矩阵 2. 从旋转矩阵转欧拉角	-	Rodrigues 公式计算旋转矩阵	$w = \cos \frac{\theta}{2}$, $\mathbf{v} = \mathbf{n} \sin \frac{\theta}{2}$
旋转矩阵	通过矩阵元素解析欧拉角	提取迹和反对称部分计算轴角: $\theta, \mathbf{n}$	-	通过矩阵元素直接解析四元数
四元数	1. 转为旋转矩阵 2. 从旋转矩阵转欧拉角	$\theta = 2 \arccos(w)$, $\mathbf{n} = [x, y, z] / \sin \frac{\theta}{2}$	使用四元数公式计算旋转矩阵	-

表中关键公式

1. 欧拉角 ↔ 旋转矩阵

- 欧拉角 → 旋转矩阵：组合单轴旋转：

$$R = R_z(\alpha)R_y(\beta)R_x(\gamma)$$

- 旋转矩阵 → 欧拉角：通过矩阵元素解析：

$$\beta = \arctan 2(-R_{31}, \sqrt{R_{11}^2 + R_{21}^2})$$

$$\alpha = \arctan 2(R_{21}, R_{11}), \quad \gamma = \arctan 2(R_{32}, R_{33})$$

2. 旋转矩阵 ↔ 轴角

- 旋转矩阵 → 轴角：

$$\theta = \arccos\left(\frac{\text{trace}(R) - 1}{2}\right)$$

$$\mathbf{n} = \frac{1}{2 \sin \theta} \begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix}$$

- 轴角 → 旋转矩阵：Rodrigues 公式：

$$R = I + \sin \theta [\mathbf{n}]_{\times} + (1 - \cos \theta) [\mathbf{n}]_{\times}^2$$

3. 轴角 ↔ 四元数

- 轴角 → 四元数：

$$w = \cos \frac{\theta}{2}, \quad x = n_x \sin \frac{\theta}{2}, \quad y = n_y \sin \frac{\theta}{2}, \quad z = n_z \sin \frac{\theta}{2}$$

- 四元数 → 轴角：

$$\theta = 2 \arccos(w), \quad \mathbf{n} = \frac{1}{\sin \frac{\theta}{2}} [x, y, z]$$

4. 旋转矩阵 ↔ 四元数

- 旋转矩阵 → 四元数：

$$w = \sqrt{1 + R_{11} + R_{22} + R_{33}}/2$$

$$x = \frac{R_{32} - R_{23}}{4w}, \quad y = \frac{R_{13} - R_{31}}{4w}, \quad z = \frac{R_{21} - R_{12}}{4w}$$

- 四元数 → 旋转矩阵：

$$R = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix}$$

注意事项

1. 旋转顺序：

- 欧拉角的旋转顺序（如 ZYX, XYZ ）对结果影响很大，必须提前明确。

2. 特殊情况：

- 旋转角为 0 时，旋转轴无意义。
- 旋转角为 π 时，可能有多个旋转轴表示。

5. 假设有一个大的 Eigen 矩阵，想把它左上角 3×3 的块取出来，然后赋值为 $\mathbf{I}_{3 \times 3}$ 。请编程实现。

```
#include <iostream>
#include <Eigen/Dense>

using namespace std;
using namespace Eigen;

#define MATRIX_SIZE 5

int main(int argc, char **argv) {
    Matrix<double, MATRIX_SIZE, MATRIX_SIZE> bigMatrix
        = MatrixXd::Random(MATRIX_SIZE, MATRIX_SIZE);

    cout << "The big matrix: \n" << bigMatrix << endl;

    Matrix3d extractedBlock = bigMatrix.block<3,3>(0,0);

    cout << "The extracted matrix block: \n" << extractedBlock << endl;

    extractedBlock.setIdentity();

    cout << "The assigned matrix block: \n" << extractedBlock << endl;

    return 0;
}
```

6. * 一般线性方程 $\mathbf{Ax} = \mathbf{b}$ 有几种做法？你能在 Eigen 中实现吗？

```
#include <iostream>
#include <Eigen/Dense>
#include <Eigen/Core>
#include <random> // 使用 C++11 随机数库

using namespace std;
using namespace Eigen;

// 自定义高斯消元法函数
VectorXd gaussianElimination(const MatrixXd& A, const VectorXd& b) {
    const Index n = A.rows(); // 矩阵的行数（即方程个数）

    // 将 A 和 b 合并为增广矩阵 [A | b]
    MatrixXd augmented(n, n + 1);
    augmented << A, b;

    // 消元阶段：将矩阵 A 转化为上三角矩阵
    for (Index k = 0; k < n; ++k) {
        // 1. 选主元：确保当前对角线元素非零
        for (Index i = k + 1; i < n; ++i) {
            if (augmented(k, k) == 0) {
                cerr << "Zero pivot encountered. Matrix is singular!" << endl;
                exit(EXIT_FAILURE);
            }
        }
        // 2. 消去第 k 列的第 i 行元素
        const double factor = augmented(i, k) / augmented(k, k);
        for (Index j = k; j <= n; ++j) {
            augmented(i, j) -= factor * augmented(k, j);
        }
    }

    // 回代阶段：从最后一行开始，求解未知数
    VectorXd x(n);
    for (Index i = n - 1; i >= 0; --i) {
        x(i) = augmented(i, n);
        for (Index j = i + 1; j < n; ++j) {
            x(i) -= augmented(i, j) * x(j);
        }
    }
}
```

```

    }
    x(i) /= augmented(i, i);
}

```

```

return x;
}

```

```

int main() {
    cout.precision(3); // 限制精确度
}

```

```

std::random_device rd; // 获取一个高质量的随机种子
std::default_random_engine generator(rd()); // 初始化随机数生成器
std::uniform_real_distribution<double> distribution(-1.0, 1.0); // 均匀分布 [-1, 1]

```

```

// 初始化 Eigen 矩阵
MatrixXd B1(3, 3); // 生成一个 rows x cols 的矩阵

```

```

for (int i = 0; i < 3; ++i) {
    for (int j = 0; j < 3; ++j) {
        B1(i, j) = distribution(generator); // 填充每个元素
    }
}

```

```

EigenSolver<Matrix3d> const solver1(B1);
MatrixXd A1 = B1.transpose() * B1 + 0.1 * MatrixXd::Identity(3, 3);

```

```

const VectorXcd eigenValueA = solver1.eigenvalues();

```

```

cout << "eigenValue of A = \n" << eigenValueA << endl;

```

```

const VectorXd b1 = VectorXd::Random(3,1);

```

```

// 1.1 调用自定义高斯消元法 (Gaussian Elimination) 函数
const VectorXd gex = gaussianElimination(A1, b1);

```

```

// 输出解
cout << "Solution of Gaussian Elimination: x = \n" << gex << endl;

```

```

// 1.2 使用 Eigen 解决 Ax=b
const VectorXd ex = A1.partialPivLu().solve(b1);

```

```
cout << "Solution of Eigen: x = \n" << ex << endl;
```

```
// 2. 使用 LU 分解法求解线性方程  $Ax=b$ 
```

```
const Matrix<double, 3, 1> lux = A1.lu().solve(b1); // A.lu().solve(b) 的 lu() 分解适用于方阵 (n×n)
```

```
cout << "Solution of lu decomposition: x = \n" << lux << endl;
```

```
// 3. 使用 LLT 分解法求解线性方程  $Ax=b$ , 要求 A 是对称正定矩阵
```

```
LLT<Matrix<double, 3, 3>> const llt(A1); // LLT 分解
```

```
if (llt.info() == Success) {
```

```
    const Matrix<double, 3, 1> lltx = llt.solve(b1);
```

```
    cout << "Solution of LLT decomposition: x = \n" << lltx << endl;
```

```
} else {
```

```
    cout << "Matrix A is not positive definite!" << endl;
```

```
}
```

```
// 4. 使用 QR 分解法
```

```
// QR 分解
```

```
HouseholderQR<MatrixXd> const qr(A1);
```

```
// 求解  $Ax = b$ 
```

```
VectorXd qrx = qr.solve(b1);
```

```
// 获取 Q 和 R
```

```
MatrixXd Q = qr.householderQ();
```

```
MatrixXd R = qr.matrixQR().triangularView<Upper>();
```

```
cout << "Solution of QR decomposition: x = \n" << qrx << endl;
```

```
cout << "Q = \n" << Q << "\nR = \n" << R << endl;
```

```
// 5. 奇异值分解 (SVD) 求解
```

```
// 对矩阵 A 进行奇异值分解
```

```
JacobiSVD<MatrixXd> const svd(A1, ComputeThinU | ComputeThinV);
```

```
// 使用 SVD 求解  $Ax = b$ 
```

```
const VectorXd svdx = svd.solve(b1);
```

```
cout << "Solution of SVD: x = \n" << svdx << endl;
```

```
// 输出奇异值
cout << "Singular values of A:\n" << svd.singularValues() << endl;
```

```
// 使用特征值分解求解
// 使用 EigenSolver 进行特征值分解
EigenSolver<Matrix3d> solver2(A1);
```

```
// 获取特征向量矩阵 V 和特征值对角矩阵 Lambda
Matrix3d V = solver2.eigenvectors().real(); // 提取实部特征向量
Vector3d Lambda = solver2.eigenvalues().real(); // 提取实部特征值
```

```
// 计算中间变量  $y = V^{-1} * b$ 
Vector3d y = V.inverse() * b1;
```

```
// 通过 Lambda 求解 y, 并还原  $x = V * y$ 
for (int i = 0; i < 3; ++i) {
    if (Lambda(i) != 0) { // 避免除以 0
        y(i) /= Lambda(i);
    } else {
        cerr << "Error: Zero eigenvalue encountered!" << endl;
        return -1;
    }
}
```

```
Vector3d evdx = V * y;
```

```
cout << "Solution of Eigen Value Decomposition: x = \n" << evdx << endl;
```

```
return 0;
```

```
}
```

第四章

1.